

Module 10 Transforming Data

IBM InfoSphere Information Server for Data Integration



Module Objectives

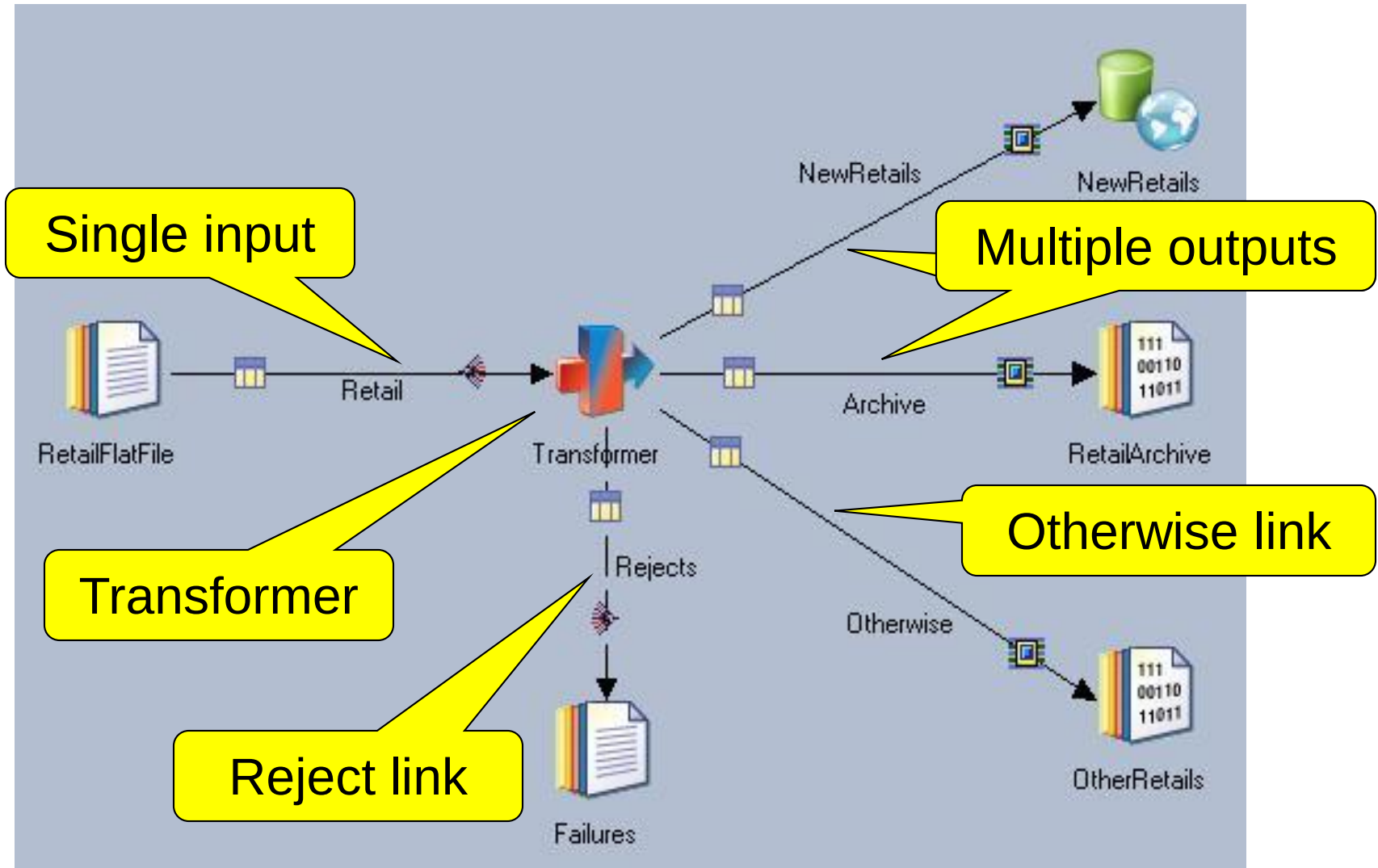
After completing this topic, you should be able to:

- Use the Transformer stage in parallel jobs
- Define constraints
- Define derivations
- Use stage variables

Transformer Stage

- Used to implement business logic
- One input link. One or more output links.
- Used to define:
 - Column mappings
 - Constraints
 - Filter data
 - Specify a condition under which rows flow out a particular output link
 - Derivations: Specify values to go into output columns
 - Can include complex IF-THEN-ELSE logic
 - An expression editor can be invoked to define the derivation
 - Derivation expressions are written using DataStage parallel transform functions

Job with a Transformer stage



Transformer Stage Variables

- Stage Variables are **evaluated in order from top to bottom**
 - Evaluated for each input row
 - Later stage variables can reference earlier stage variables
 - Earlier stage variables can reference later stage variables
 - These variables will contain a value derived from the previous row that came into the Transformer
- Multi-purpose
 - Counters
 - Store values from previously read rows to make comparisons with the currently read row
 - Store derived values to be used in multiple target field derivations
 - Can be used to control execution of constraints

Transformer Stage Editor

Transformer_24 - Transformer Stage

Stage Variables

Derivation	Stage Variable
SaveInputRecord()	NumSavedRows
LastRowInGroup(inlink.PROD_CATEGORY_ID)	IsBreak
IF IsBreak THEN SummingPrice+ inlink.PRICE ELSE 0	TotalPrice
IF IsBreak THEN 0 ELSE SummingPrice + inlink.PRICE	SummingPrice
IF IsBreak THEN NumSavedRows ELSE 0	NumRows

Loop Condition

Loop While: @ITERATION <= NumRows

Derivation	Loop Variable
GetSavedInputRecord()	SavedRowIndex

Constraints

Input columns

Derivations

Output columns

Columns definitions

Column Name	Derivation
PROD_CATEGORY_ID	inlink.PROD_CATEGORY_ID
DESCRIPTION	inlink.DESCRPTION
PRICE	inlink.PRICE
Percentage	(inlink.PRICE * 100) / TotalPrice

inlink

	Column name	Key	SQL type	Extended	Length	Scale	N
1	PROD_CATEGORY_ID	☐	VarChar	Unicode	255		N
2	DESCRIPTION	☐	VarChar	Unicode	255		N
3	PRICE	☐	Numeric		8	2	N
4	DATE	☐	Date				Y

outlink

	Column name	Key	SQL type	Extended	Length	Scale	Nullable	Description
1	PROD_CATEGORY_ID	☐	VarChar	Unicode	255		No	
2	DESCRIPTION	☐	VarChar	Unicode	255		No	
3	PRICE	☐	Numeric		8	2	No	
4	Percentage	☐	Numeric		8	2	Yes	

OK Cancel Help

Defining a Derivation

The screenshot shows a software interface for defining a derivation. It features a table with columns for 'Constraint', 'Derivation', and 'Column Name'. The 'Derivation' column contains several lines of code. A dropdown menu is open on the left, showing various options for building the derivation expression. Yellow callout boxes with arrows point to specific parts of the interface to highlight key concepts:

- Input column:** Points to the text `inlink.PROD_CATEGORY_ID` in the first row of the derivation table.
- IF - THEN - ELSE:** Points to the conditional logic `IF inlink.PRICE > 100 THEN inlink.PRICE * 0.8 ELSE inlink.PRICE` in the second row.
- Stage variable:** Points to the expression `(inlink.PRICE * 100) / TotalPrice` in the third row.
- String concatenation:** Points to the text `'CAT ': inlink.DESCRPTION : 'A'` in the fourth row.
- Function:** Points to the `CurrentDate()` function in the fifth row.

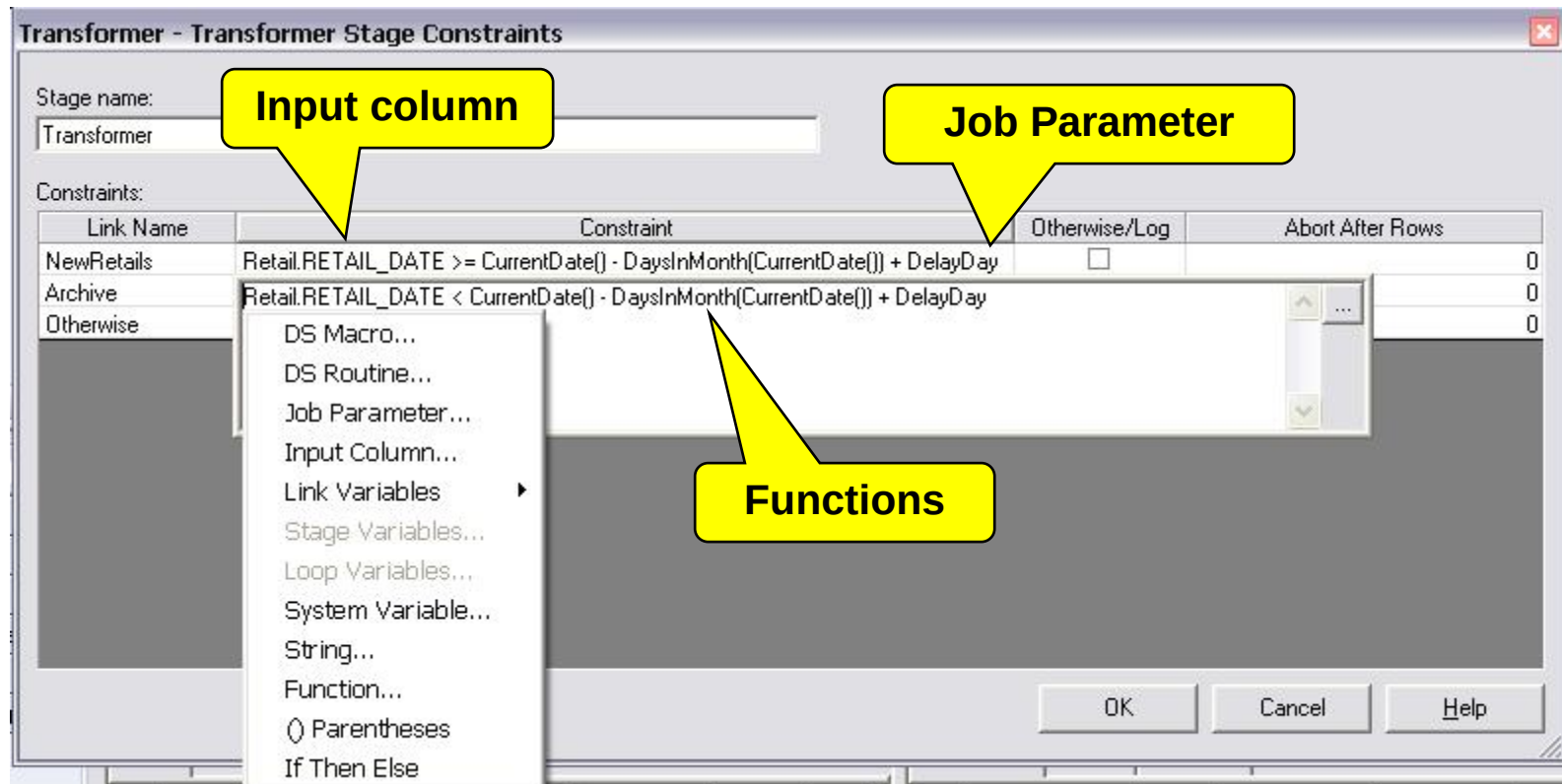
The dropdown menu on the left includes the following options:

- DS Macro...
- DS Routine...
- Job Parameter...
- Input Column...
- Stage Variables...
- Loop Variables...
- System Variable...
- String...
- Function...
- () Parentheses
- If Then Else

Constraint:	Derivation	Column Name
	<code>inlink.PROD_CATEGORY_ID</code>	<code>PROD_CATEGORY_ID</code>
	<code>IF inlink.PRICE > 100 THEN inlink.PRICE * 0.8 ELSE inlink.PRICE</code>	<code>PRICE</code>
	<code>(inlink.PRICE * 100) / TotalPrice</code>	<code>PERCENTAGE</code>
	<code>'CAT ': inlink.DESCRPTION : 'A'</code>	<code>ON</code>
	<code>CurrentDate()</code>	<code>DATE_INS</code>

Defining a Constraint

- A constraint defines a filter condition for each output link
- “Otherwise” link collects the rows which failed all the constraints conditions



Defining Local Stage Variables

The diagram illustrates the components of a Stage Variables table. A yellow callout labeled 'Input column' points to the 'DSLink29.PROD_CATEGORY_ID' input in the 'LastRowInGroup' function. Another yellow callout labeled 'Functions' points to the 'SaveInputRecord()' function in the first row. A third yellow callout labeled 'Variable' points to the 'NumSavedRows' variable in the last row.

Stage Variables	
Derivation	Stage Variable
SaveInputRecord()	NumSavedRows
LastRowInGroup(DSLink29.PROD_CATEGORY_ID)	IsBreak
IF IsBreak THEN SummingPrice+ DSLink29.PRICE ELSE 0	TotalPrice
IF IsBreak THEN 0 ELSE SummingPrice + DSLink29.PRICE	SummingPrice
IF IsBreak THEN NumSavedRows ELSE 0	NumRows

IF – THEN – ELSE Derivation

- Use IF THEN ELSE to conditionally derive a value
- Syntax: IF <condition> THEN <expression1> ELSE <expression2>
- The result of “expression1” (“expression2”) is copied to the target column if “condition” is TRUE (FALSE)
- Example:

outlink	
Constraint:	
Derivation	Column Name
inlink.PROD_CATEGORY_ID	PROD_CATEGORY_ID
IF inlink.PRICE > 100 THEN inlink.PRICE* 0.8 ELSE inlink.PRICE	PRICE

inlink.PRICE	outlink.PRICE
10	10
1000	800

Transformer Functions

- **Date & Time:** perform various operations on dates and times
- **Last Row Handling:** detect key break when looping
- **Logical:** perform bit operations
- **Mathematical:** perform mathematical operations
- **Null Handling:** allow NULL in derivations
- **Number:** extract from decimal or floating point numbers
- **Raw:** obtain the length of the data in a column containing raw data
- **String:** manipulate strings
- **Type Conversion:** change the type of an argument
- **Utility:** variety of different purposes
- **Vector:** access an element in a vector column

String Functions and Operators

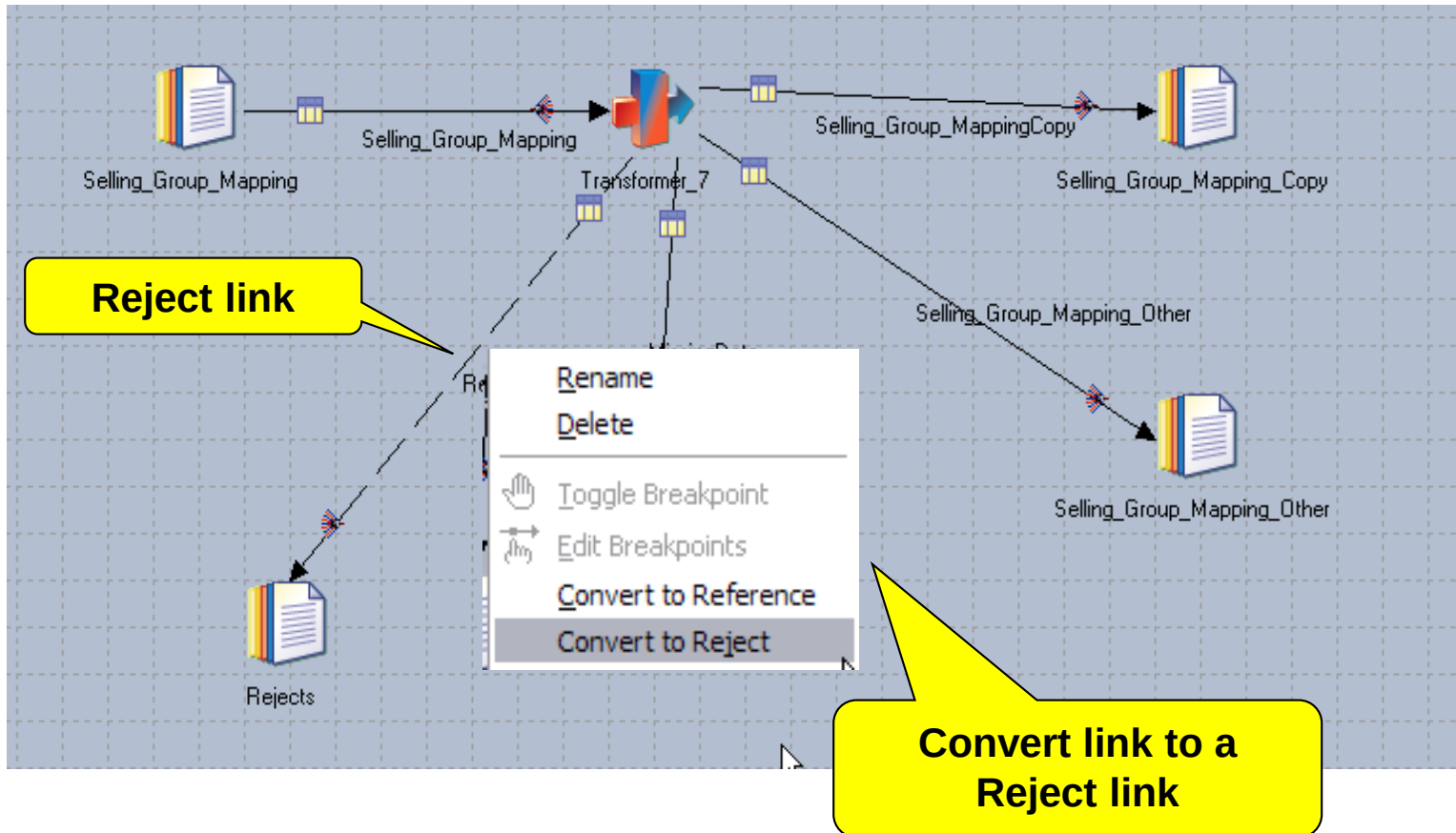
- Substring Operator : returns a part of a string
 - `<string>[loc,length]`
- UpCase / DownCase : returns the string in uppercase or lowercase
 - `UpCase(<string>) / DownCase(<string>)`
- Len : returns the length of a string in characters
 - `Len(<string>)`
- Trim : removes the trailing and leading spaces and tab
 - `Trim(<string>)`
- Example:

STRING TEST	STRING[2,3]	UpCase(STRING)	LEN(STRING)	TRIM(STRING)
" abcde "	"abc"	"ABCDE "	7	"abcde"

Checking for NULLs

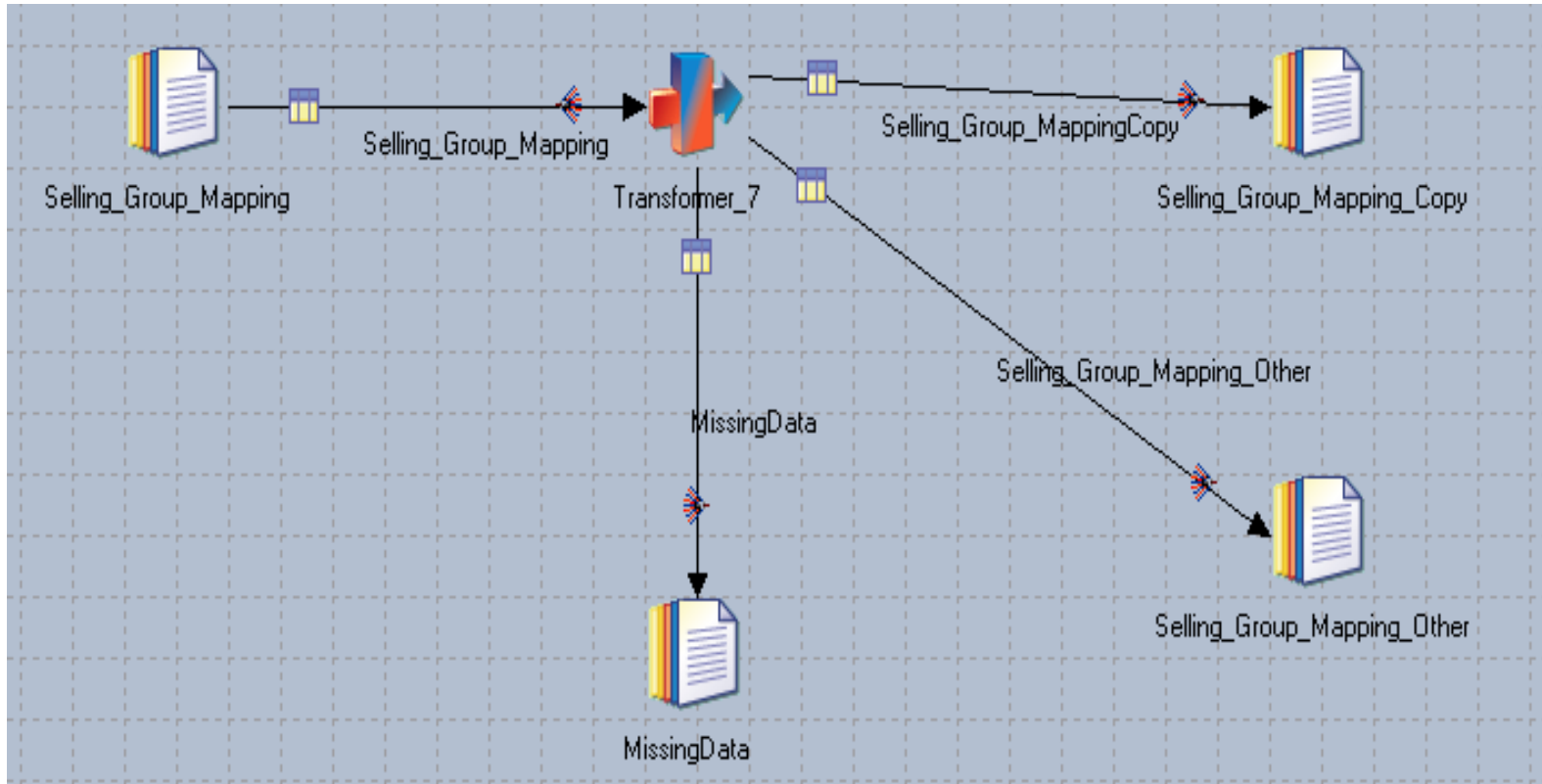
- NULL is allowed (starting with Version 8.5) and can be handled in constraints, derivations, stage variables
- NULL functions
 - Testing for NULL
 - IsNull(<column>)
 - IsNotNull(<column>)
- Replace NULL with a value
 - NullToValue(<column>, <value>)
- Set a derivation to NULL: SetNull()

Transformer Reject Link



- The reject link takes rows that have not been written to any other links because of write failure or expression evaluation failure. Not part of the constraints tab inside the stage but defined outside of stage.

Otherwise Link



- The otherwise link takes rows that have not been written to any other links because of failure to meet constraints on all the previous output links. Defined in the constraints tab.

Defining an Otherwise Link

Transformer_7 - Transformer Stage Constraints

Stage name:
Transformer_7

Constraints:

Link Name	Constraint	Otherwise/Log	Abort After Rows
Selling_Group_MappingCopy	UpCase(Selling_Group_Mapping.Distribution_Channel_Description) = UpCase(C	☐	0
Selling_Group_Mapping_Other	UpCase(Selling_Group_Mapping.Distribution_Channel_Description) <> UpCase	☐	0
MissingData		☒	50

Must come last after the other links.

Check to create otherwise condition

Can specify abort condition

A warning is put in the log if any rows go down the otherwise link.

Specifying Link Ordering

